

SESIÓN 5

GameStore Escolar

PASO 1 — Proyecto en VS Code

Crear una carpeta:

Sesion5

Dentro quedarán:

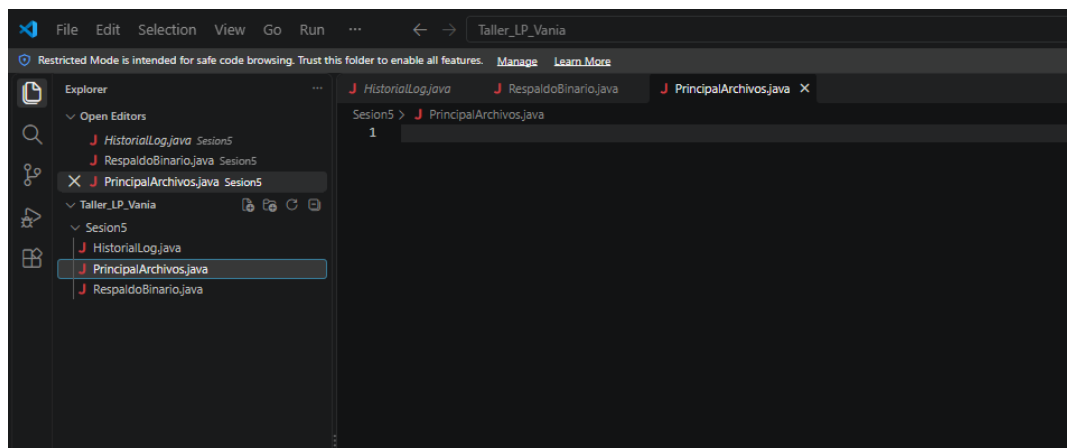
Guia5_Archivos_Java

|

├── HistorialLog.java

├── RespaldoBinario.java

└── PrincipalArchivos.java



PASO 2 — HistorialLog.java

```
1 import java.io.*;
2 import java.io.*;
3 import java.io.*;
4 import java.io.*;
5 import java.io.*;
6
7 public class HistorialLog {
8
9     // Metodo para guardar un registro en archivo de texto (Trabajo 1)
10    public static void registrarCanje(String estudiante, String itemCanjeado) {
11        // Uso de FileWriter en modo append (true) para no sobrescribir el archivo
12        try (PrintWriter pw = new PrintWriter(new FileWriter("historial_canjes.txt", true))) {
13            pw.println("Estudiante: " + estudiante + " | Item: " + itemCanjeado);
14            System.out.println("Registro de canje guardado exitosamente en el archivo de texto.");
15        } catch (IOException e) {
16            System.out.println("Error de E/S al escribir en el archivo de texto: " + e.getMessage());
17        }
18    }
19
20    // Metodo para leer el historial de texto
21    public static void leerHistorial() {
22        try (BufferedReader br = new BufferedReader(new FileReader("historial_canjes.txt"))) {
23            String linea;
24            System.out.println("--- HISTORIAL DE CANJES (TEXT-FILE) ---");
25            while ((linea = br.readLine()) != null) {
26                System.out.println(linea);
27            }
28        } catch (IOException e) {
29            System.out.println("No se encontro el archivo o hubo un error al leerlo: " + e.getMessage());
30        }
31    }
32 }
```

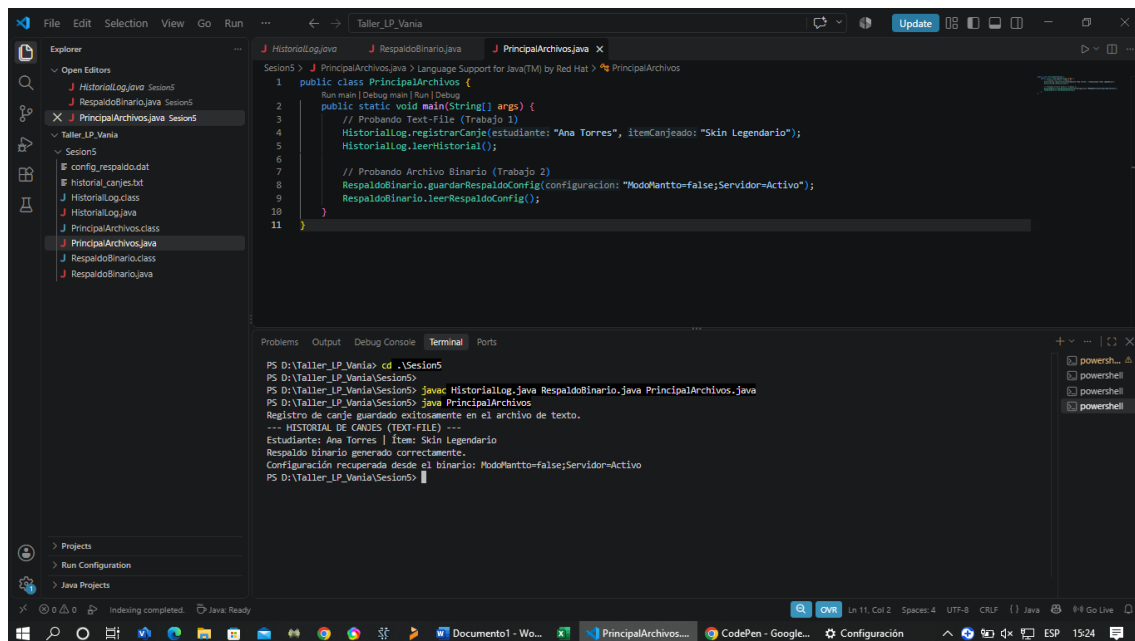
PASO 3 — RespaldoBinario.java

```
1 import java.io.*;
2 import java.io.*;
3 import java.io.*;
4 import java.io.*;
5 import java.io.*;
6
7 public class RespaldoBinario {
8
9     // Metodo para guardar un objeto de configuraci3n en archivo binario (Trabajo 2)
10    public static void guardarRespaldoConfig(String configuracion) {
11        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("config_respaldo.dat"))) {
12            oos.writeObject(configuracion);
13            System.out.println("Respaldo binario generado correctamente.");
14        } catch (IOException e) {
15            System.out.println("Error al escribir el archivo binario: " + e.getMessage());
16        }
17    }
18
19    // Metodo para leer el respaldo binario
20    public static void leerRespaldoConfig() {
21        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("config_respaldo.dat"))) {
22            String configLeida = (String) ois.readObject();
23            System.out.println("Configuraci3n recuperada desde el binario: " + configLeida);
24        } catch (IOException | ClassNotFoundException e) {
25            System.out.println("Error al leer el archivo binario: " + e.getMessage());
26        }
27    }
28 }
```

PASO 4 — PrincipalArchivos.java

```
1 public class PrincipalArchivos {
2     public static void main(String[] args) {
3         // Probando Text-File (Trabajo 1)
4         HistorialLog.registrarCanje("Ana Torres", "Skin Legendario");
5         HistorialLog.leerHistorial();
6
7         // Probando Archivo Binario (Trabajo 2)
8         RespaldoBinario.guardarRespaldoConfig("ModoMantto=False;Servidor=Activo");
9         RespaldoBinario.leerRespaldoConfig();
10    }
11 }
```

PASO 5 — Ejecutar el programa



Sustentación Conceptual

Es obligatorio usar try-catch o throws porque las operaciones de lectura y escritura de archivos pueden generar excepciones, como errores de acceso, archivos inexistentes o problemas de E/S. El try-catch permite controlar estos errores y evitar que el programa termine inesperadamente.